CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 17: Codebase Exploration

> **Exam objective:** Apply systematic codebase exploration strategies using Grep, Glob, and Read tools that build incremental understanding while managing context window constraints.

Q1.

You have just opened an unfamiliar repository and need to understand where authentication is implemented without flooding the context window. What is the best first step?

A. Read every file under src\ recursively
B. Use Glob to identify likely auth-related files, then Grep for key symbols, then Read only the most relevant files
C. Start by opening the largest configuration file because it may reference auth
D. Search the entire repo for every occurrence of "if" and "login" together

Answer: B
Explanation: Effective exploration narrows scope incrementally: find candidate files, locate relevant symbols, and then read targeted content. Reading everything wastes context, and overly broad searches produce noisy results with poor information gain.

Q2.

You suspect a bug originates in code that constructs database connection strings, but you do not know the file names. Which sequence is most efficient?

A. Read all YAML files first
B. Grep for "connection string" across the entire repository and then stop
C. Glob for likely config and service files, Grep for connection-related terms, then Read matched implementations
D. Open the README and infer the implementation from architecture diagrams

Answer: C
Explanation: Combining Glob and Grep creates a focused candidate set before consuming context on actual file reads. A README may help later, but targeted search is the best path to precise implementation details.

Q3.

When investigating how requests flow through an API, what is the main benefit of reading only the controller, service, and repository files that Grep identified?

A. It guarantees complete understanding of the entire system
B. It preserves context by following the most relevant execution path instead of reading unrelated modules
C. It avoids the need to use search tools at all
D. It ensures no symbols are missed anywhere in the repo

Answer: B
Explanation: Following the most relevant path keeps the investigation coherent and context-efficient. It does not guarantee total coverage, but it is the right strategy for building layered understanding without drowning in unrelated code.

Q4.

You used Grep to find 120 matches for "cache" across the repo. What is the best next move?

A. Read all 120 matched files in full
B. Abandon the investigation because the term is too common
C. Refine the search using more specific terms, file globs, or architectural clues before reading files
D. Replace Grep with a random sample of five files

Answer: C
Explanation: High-result searches should be narrowed before reading, using more precise symbols, path filters, or related identifiers. Reading everything would waste context and likely obscure the important implementation details.

Q5.

You need to understand a build failure involving TypeScript path aliases. Which approach best balances speed and context control?

A. Read every tsconfig variant and every import in the repo
B. Glob for tsconfig files, read those first, then Grep for the problematic alias and inspect the highest-value matches
C. Search only package.json because aliases cannot be defined elsewhere
D. Start with test files since failures often surface there

Answer: B
Explanation: Configuration issues are best approached by locating the relevant config files first and then tracing actual usage of the alias. This sequence minimizes unnecessary reads while preserving enough context to reason about cause and effect.

Q6.

What is the strongest reason to prefer incremental reading over opening large directories wholesale?

A. Incremental reading is slower, which improves accuracy
B. Incremental reading reduces the chance of context-window saturation and keeps the current mental model focused

C. Large-directory reading is unsupported
D. Incremental reading eliminates the need to revisit files later

Answer: B
Explanation: Context is finite, so targeted reading helps retain the most relevant facts and relationships. It does not prevent revisits, but it does reduce context churn and improves the clarity of the evolving model.

Q7.

You are tracing feature-flag behavior. After Grep finds the flag definition and several call sites, what is the best reading order?

A. Open all call sites alphabetically
B. Read the feature-flag library first, then the call site most connected to the reported behavior, then follow downstream references
C. Ignore the definition because only usage matters
D. Read only tests because they summarize intent

Answer: B
Explanation: Starting with the definition clarifies semantics, then reading the most relevant call path grounds the investigation in the observed behavior. Alphabetical order and test-only approaches are less likely to build a correct causal model.

Q8.

Which exploration pattern is most likely to waste context with little payoff?

A. Reading a file immediately after Grep identifies a high-signal symbol
B. Using Glob to separate application code from generated code
C. Opening dozens of low-relevance files because they share a vague keyword
D. Reading a config file before tracing runtime behavior when the bug appears configuration-related

Answer: C
Explanation: Vague keyword matches often pull in tangential references that do not advance the investigation. The other options are targeted and typically produce meaningful information relative to the context they consume.

Q9.

You need to learn how a CLI command is wired from entrypoint to handler. Which strategy is best?

A. Grep for the command name or parser registration, read the entrypoint, then follow linked handler files
B. Read all shell scripts in the repository
C. Search for "main" and read every result
D. Start with dependency lockfiles to infer the CLI framework

Answer: A
Explanation: CLI wiring is usually discoverable from registration or parser definitions, making

targeted search the most efficient starting point. The other approaches are noisy and indirect.

Q10.

During a long exploration, you notice that earlier discoveries about module boundaries are slipping out of active context. What is the best corrective action?

A. Re-read the entire repository from the beginning
B. Use a concise scratch summary and continue targeted reading from the current frontier
C. Stop using Grep because it fragments attention
D. Open only binary assets to reset the context

Answer: B
Explanation: A compact scratch summary preserves key findings while letting you continue reading selectively. Re-reading everything is inefficient, and abandoning search tools would make the exploration less systematic, not more.

Q11.

You are exploring a polyglot repo with generated client SDKs that overwhelm search results. What is the best first adjustment?

A. Search the generated directories first because they are comprehensive
B. Use Glob or path filters to exclude generated code from early exploration
C. Read every generated file once so you can safely ignore them later
D. Switch directly to manual browsing without search tools

Answer: B
Explanation: Excluding generated artifacts early reduces noise and keeps attention on source-of-truth code. Generated outputs are often useful later for verification, but they are poor starting points for architectural understanding.

Q12.

Why is it usually better to Read a few high-value file sections than entire long files when exploring?

A. Section reads help focus on the relevant symbols and preserve context for follow-up reasoning
B. Full-file reads are always prohibited
C. Section reads guarantee you will never miss dependencies
D. Shorter reads automatically update documentation

Answer: A
Explanation: Focused reads capture the exact lines that matter while leaving room in context for related evidence and synthesis. They do not guarantee completeness, but they improve the signal-to-noise ratio substantially.

Q13.

You need to identify whether retry logic lives in middleware or in the HTTP client itself. Which plan is best?

A. Grep for "retry", read the middleware pipeline registration, and compare with HTTP client wrappers matched by search
B. Read all markdown files for the word "resilience"
C. Start from unit tests only and avoid implementation files
D. Search for all numbers greater than 3, since retry counts are often small integers

Answer: A
Explanation: The question is architectural, so the best approach is to inspect both likely enforcement layers identified through targeted search. The distractors are indirect and likely to miss the actual implementation boundary.

Q14.

What does "building incremental understanding" most nearly mean in codebase exploration?

A. Accumulating every possible fact before forming any hypothesis
B. Forming and updating a working model as each search and read step narrows uncertainty
C. Reading files in filesystem order to avoid bias
D. Letting context fill with raw code until patterns emerge automatically

Answer: B
Explanation: Good exploration is iterative: each result refines the hypothesis and guides the next targeted search or read. Exhaustive accumulation is rarely feasible and usually harms context discipline.

Q15.

Which workflow best demonstrates context-aware exploration discipline?

A. Glob → Grep → targeted Read → concise notes → follow next symbol chain
B. Read all source files → Grep later if confused
C. Grep broad common terms repeatedly without refining scope
D. Open random files until one seems relevant

Answer: A
Explanation: This workflow creates a controlled funnel from broad discovery to precise understanding, while preserving room for synthesis. The other options either over-consume context or rely on unsystematic browsing.
